

PRM Framework: Process Ancestry-Based Security for HPC Environments

Abstract

Modern High-Performance Computing (HPC) environments face security challenges balancing protection with performance. This paper presents the PRM Framework, a process ancestry-based security solution for Linux that provides fine-grained control over system calls with minimal overhead (0.5-2% vs. SELinux's 5-10%).

1 Introduction

Security in HPC environments requires minimal overhead while addressing containerization risks. Traditional solutions like SELinux and AppArmor introduce performance penalties or lack context awareness.

1.1 Contributions

- Novel process ancestry-based security with minimal performance impact
- Comprehensive architecture for efficient system call filtering
- Performance benchmarks showing significant advantages over traditional solutions
- Flexible policy framework for process ancestry-based rules

2 Background

2.1 Linux Security Mechanisms

Current Linux security includes DAC, MAC (SELinux/AppArmor), capabilities, namespaces, and seccomp.

2.2 HPC Security Challenges

HPC environments face unique challenges: performance sensitivity, complex workflows requiring privileges, multi-tenancy, and containerization risks.

3 Related Work

Prior work includes Systrace, Seccomp-BPF, SPEAKER, Docker Security, gVisor, and HPC-specific solutions.

4 Methodology

4.1 Research Questions

We address how to provide robust security without performance overhead, using process ancestry for context.

4.2 Requirements

Functional requirements include system call filtering based on process ancestry. Non-functional requirements include minimal overhead (i3%) and compatibility with existing systems.

5 PRM Framework Design

5.1 Architecture Overview

The PRM Framework consists of:

- System Call Interception
- Cache Mechanism
- Filters
- Verifier (Process Ancestry Checker)
- PRM Table Rules
- PROG Handlers

5.2 Process Ancestry Analysis

The framework extracts command names from current process, parent, and grandparent to make security decisions.

6 Evaluation

6.1 Performance Comparison

PRM Framework shows 0.5-2% system call overhead compared to SELinux (5-10%) and AppArmor (3-7%).

6.2 Performance Benchmarks

Benchmarks on LINPACK, STREAM, IOR, and IMB-MPI show minimal impact on HPC workloads.

6.3 Security Effectiveness

The framework successfully prevented container escape attempts and privilege escalation.

7 Discussion

7.1 Limitations

Current limitations include configuration complexity and lack of integration with SIEM systems.

8 Conclusion

The PRM Framework provides a novel approach to Linux security well-suited for HPC environments, with future work focusing on automated rule generation and machine learning integration.